

INDIAN STATISTICAL INSTITUTE, KOLKATA
BACHELOR OF STATISTICS



Statistical Methods IV

Semester Project

Prediction and Analysis of Hepatitis-C

17TH April 2026

Anushruta Pakhira (BS2407)
Pujari Supriya (BS2027)
Liza Parvin (BS2417)

Advisor/Instructor: Prof. Shyamal Krishna De

Contents

1	Introduction	2
1.1	Aim of the Project	2
1.2	Data Description	2
1.2.1	Covariates	2
1.2.2	Data Sample	2
2	Theoretical Background of the Ordinal Logistic regression models	3
2.1	Traditional Proportional Odds Approach	3
2.2	Estimation of βs and αs	4
3	Fitting and Testing Model using R-Code	5
3.1	Fitting the Model	5
3.2	Summary of Model	8
3.3	Confusion Heat Matrix	10
4	Conclusion	11
5	Complete R Code	13

1 Introduction

1.1 Aim of the Project

In this project, we aim to predict if a person has Hepatitis C or not and if they have Hepatitis C, what is its severity, using a proper logistic model.

Our data contains biological factors, like cholesterol, age, GGT etc. The data was collected from Kaggle. The data can be found at the following link : [Hepatitis-C](#)

1.2 Data Description

1.2.1 Covariates

- Category : Stage of Hepatitis C
- Age : Age of the subject
- ALB :Albumin
- ALP : Alkaline Phosphatase
- ALT : Alanine Aminotransferase
- AST : Aspartate Aminotransferase
- BIL : Bilirubin
- CHE : Cholinesterase
- CHOL : Cholesterol
- CREA : Creatinine
- GGT : Gamma-Glutamyl Transferase
- PROT : Total amount of Protein in blood (Albumin + Globulin)

1.2.2 Data Sample

```
> head(data)
```

	Category	Age	ALB	ALP	ALT	AST	BIL	CHE	CHOL	CREA	GGT	PROT
1	0=Blood Donor	32	38.5	52.5	7.7	22.1	7.5	6.93	3.23	106	12.1	69.0
2	0=Blood Donor	32	38.5	70.3	18.0	24.7	3.9	11.17	4.80	74	15.6	76.5
3	0=Blood Donor	32	46.9	74.7	36.2	52.6	6.1	8.84	5.20	86	33.2	79.3
4	0=Blood Donor	32	43.2	52.0	30.6	22.6	18.9	7.33	4.74	80	33.8	75.7
5	0=Blood Donor	32	39.2	74.1	32.6	24.8	9.6	9.15	4.32	76	29.9	68.7
6	0=Blood Donor	32	41.6	43.3	18.5	19.7	12.3	9.92	6.05	111	91.0	74.0

2 Theoretical Background of the Ordinal Logistic regression models

Our target variable (Category) consists of stages of Hepatitis C

Blood Donor $\rightarrow$ Hepatitis C $\rightarrow$ Fibrosis $\rightarrow$ Cirrhosis

There is an order in the levels of the Category, thus, rather than using multinomial logistic regression (which is mostly used for nominal categories), it would be better to use ordinal logistic regression. And among ordinal logistic regression, the Traditional Proportional Odds Approach is widely used.

2.1 Traditional Proportional Odds Approach

In ordinal logistic models, the dependent variable 'Y' manifests in a latent form with a sense of order between its categories. Therefore, Y represents an ordinal categorical variable, and estimations give the probability of occurrence of each of its respective m levels/categories ($m \geq 2$).

As explained by Hilbe, within the framework of the proportional odds model, there is a comparison between the probability of an equal or lower response ($Y \leq m$) and the probability of a higher response ($Y > m$). Agresti points out that estimations via proportional odds generate models that concomitantly use the logits (Z) of cumulative probabilities, as shown in the following equation:

$$P(Y \leq m | X) = P_{Y=1}(X) + P_{Y=2}(X) + \dots + P_{Y=m}(X) \quad (1)$$

being $m = 1, \dots, M$.

Following Agresti, cumulative logits are defined as

$$\text{logit}(P(Y \leq m | X)) = \log \left(\frac{P(Y \leq m | X)}{1 - P(Y \leq m | X)} \right) \quad (2)$$

$$= \log \left(\frac{P_{Y=1}(X) + P_{Y=2}(X) + \dots + P_{Y=m}(X)}{P_{Y=m+1}(X) + P_{Y=m+2}(X) + \dots + P_{Y=M}(X)} \right) \quad (3)$$

where $m = 1, \dots, M - 1$, and each logit Z uses all the m response categories of Y .

From the above equation, the proportional odds model can be described mathematically as follows :

$$\text{logit}(P(Y \leq m | X)) = \alpha_m - \beta'X \quad (4)$$

where $m = 1, \dots, M - 1$.

The application of the above yields identical slope coefficients (β) for each response level

of Y , but with different angular coefficients (a_m) depending on the change in the category studied.

2.2 Estimation of β s and α s

In our data set the target variable(*Category*) has 4 levels. And the 4 levels are factored as, 0 if *Healthy/Blood Donor*, 1 if *Hepatitis C*, 2 if *Fibrosis* and 3 if *Cirrhosis*. Thus our dependent variable $Y_i \in \{0, 1, 2, 3\}$, let the corresponding predictors be $X_i = (x_{i1}, \dots, x_{ip})$. Let, $\beta = (\beta_1, \dots, \beta_p)$ and $\eta_i = \beta^T X_i$.

From the equation (4), the following can be written.

$$P(Y_i \leq j \mid X_i) = \frac{1}{1 + \exp(-(\alpha_j - \eta_i))} = \frac{\exp(\alpha_j - \eta_i)}{1 + \exp(\alpha_j - \eta_i)} \quad (5)$$

Define $\pi_{ij} := P(Y_i \leq j \mid X_i) = \frac{1}{1 + e^{-(\alpha_j - \eta_i)}}$.

Therefore, the category probabilities can be written as:

$$P(Y_i = 0 \mid X_i) = \pi_{i0}$$

$$P(Y_i = 1 \mid X_i) = \pi_{i1} - \pi_{i0}$$

$$P(Y_i = 2 \mid X_i) = \pi_{i2} - \pi_{i1}$$

$$P(Y_i = 3 \mid X_i) = 1 - \pi_{i2}$$

Now, to estimate the β s and α s, we need to maximize the log-likelihood function. Where the likelihood function is given as

$$L(\alpha, \beta) = \prod_{i=1}^n P(Y_i = y_i \mid X_i) \quad (6)$$

```
> table(data$Category)

 0    1    2    3 
533  20  12  24 

> prop.table(table(data$Category))

      0      1      2      3 
0.90492360 0.03395586 0.02037351 0.04074703
```

But since our dataset is very imbalanced, i.e. 90% of the Category consists of Healthy people, thus even if a model predicts all the data to 0 (ie. Healthy/no-hepatitis) we will have 90% accuracy, thus such a model is misleading, so to deal with this it's better

to use the Weighted Likelihood Function instead of Likelihood function. The Weighted Likelihood Function is given by

$$L(\alpha, \beta) = \prod_{i=1}^n [P(Y_i = y_i | X_i)]^{w_i}$$

In the article "Class-Weighted Classification: Trade-offs and Robust Approaches", where for each class they choose specific weights to address each class properly, and in our model, we want to give higher importance to rare classes and lower importance to more frequent classes, thereby helping to address class imbalance. Also in "A Machine Learning Based Diabetes Mellitus Classification on Weighted Class using Inverse Class Frequency Method" they used *Inverse class frequencies* method for assigning weights to prevalent classes in the dataset. Thus we choose the following weight function, $w_i = \frac{1}{(\frac{1}{N} \sum \frac{1}{n_{y_i}})}$, where n_{y_i} is the frequency of class y_i and N is total number of observations.

The weighted log - likelihood function is :

$$\ell(\alpha, \beta) = \sum_{i=1}^n w_i \log P(Y_i = y_i | X_i)$$

and by expanding it further, we will get

$$\ell(\alpha, \beta) = \sum_{i:y_i=0} w_i \log \pi_{i0} + \sum_{i:y_i=1} w_i \log(\pi_{i1} - \pi_{i0}) + \sum_{i:y_i=2} w_i \log(\pi_{i2} - \pi_{i1}) + \sum_{i:y_i=3} w_i \log(1 - \pi_{i2})$$

The parameters (α, β) are estimated by solving:

$$\frac{\partial \ell}{\partial \alpha_j} = 0, \quad \frac{\partial \ell}{\partial \beta} = 0$$

using iterative methods such as Fisher Scoring or Newton-Raphson.

3 Fitting and Testing Model using R-Code

3.1 Fitting the Model

At first we omit the incomplete observations and add the *Category_num* column (changing the categories to their factors), then delete the original Category column and scale the predictor variables. And split the data into Training and testing sets (75% split).

```

raw_data <- read.csv("hcvdat0.csv")
data <- na.omit(raw_data)

data$Category_num <- as.numeric(factor(data$Category,
                                     levels = c("0=BloodDonor", "1=Hepatitis C",
                                     "2=Fibrosis", "3=Cirrhosis")) - 1

data<- data[ , -1]
data[, -12]<- scale(data[, -12])

split <- sample.split(data$Category_num, SplitRatio = 0.75)
X_tr <- subset(data, split == "TRUE")
X_ts <- subset(data, split == "FALSE")

```

The next step is to train and fit the proportional odds model.

```

class_counts <- table(X_tr$Category_num)
wts <- as.numeric(1 / class_counts[as.character(X_tr$Category_num)])
wts <- wts / mean(wts)

model <- vglm(Category_num ~ Age + ALB + ALT + AST + BIL +
              CHE + CHOL + CREA + GGT + ALP + PROT,
              family = cumulative(parallel = TRUE),
              data = X_tr,
              weights = wts)

```

Now, looking at the summary, we can figure out the important predictors and improve our model accordingly

```
> summary(model)
```

Call:

```
vglm(formula = Category_num ~ Age + ALB + ALT + AST + BIL + CHE +  
      CHOL + CREA + GGT + ALP + PROT, family = cumulative(parallel = TRUE),  
      data = X_tr, weights = weights)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept):1	0.15975	0.17826	0.896	0.370166
(Intercept):2	2.40910	0.23381	10.304	< 2e-16 ***
(Intercept):3	5.21249	0.34697	15.023	< 2e-16 ***
Age	-0.33685	0.10996	-3.063	0.002188 **
ALB	0.51990	0.15720	3.307	0.000942 ***
ALT	1.56195	0.21911	7.129	1.01e-12 ***
AST	-0.80145	0.11444	-7.003	2.50e-12 ***
BIL	-0.89238	0.20983	-4.253	2.11e-05 ***
CHE	0.07310	0.11610	0.630	0.528907
CHOL	0.68204	0.14402	4.736	2.18e-06 ***
CREA	-0.51231	0.11170	-4.587	4.50e-06 ***
GGT	-0.10129	0.07814	-1.296	0.194858
ALP	0.11388	0.08213	1.387	0.165585
PROT	-0.47086	0.11515	-4.089	4.33e-05 ***

As the p-value of *CHE*, *GGT*, *ALP* is ≥ 0.05 remove it to make the model better.

```
model_fixed <- vglm(Category_num ~ Age + ALB + ALT + AST + BIL +  
                    CHOL + CREA + PROT ,  
                    family = cumulative(parallel = TRUE),  
                    data = X_tr,  
                    weights = wts)
```


3.2 Summary of Model

```
> summary(model_fixed)
```

Call:

```
vglm(formula = Category_num ~ Age + ALB + ALT + AST + BIL + CREA +  
      PROT + CHOL, family = cumulative(parallel = TRUE), data = X_tr,  
      weights = weights)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept):1	0.1310	0.1754	0.747	0.455086
(Intercept):2	2.3633	0.2290	10.319	< 2e-16 ***
(Intercept):3	5.1717	0.3443	15.021	< 2e-16 ***
Age	-0.3686	0.1052	-3.503	0.000459 ***
ALB	0.4734	0.1383	3.423	0.000618 ***
ALT	1.6268	0.2121	7.669	1.73e-14 ***
AST	-0.8725	0.1009	-8.650	< 2e-16 ***
BIL	-0.8660	0.1883	-4.598	4.26e-06 ***
CREA	-0.4954	0.1137	-4.358	1.32e-05 ***
PROT	-0.4650	0.1129	-4.118	3.82e-05 ***
CHOL	0.7124	0.1296	5.498	3.84e-08 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Names of linear predictors: logitlink(P[Y<=1]), logitlink(P[Y<=2]),
logitlink(P[Y<=3])

Residual deviance: 732.6049 on 1315 degrees of freedom

Log-likelihood: -366.3024 on 1315 degrees of freedom

Number of Fisher scoring iterations: 9

Exponentiated coefficients:

Age	ALB	ALT	AST	BIL	CREA
0.6916935	1.6054288	5.0876349	0.4178986	0.4206348	0.6093284
PROT	CHOL				
0.6281492	2.0388373				

The residual deviance is 732.60 on 1315 degrees of freedom ($732.60/1315 = 0.5571103$), suggesting an adequate overall fit to the data.

The exponentiated coefficients represent $e^{\beta_j} = \frac{\text{odds}(Y > k | x_j + 1)}{\text{odds}(Y > k | x_j)}$. so values greater than 1 increase the likelihood of higher severity, while values less than 1 decrease it. ALT (OR ≈ 5.09), cholesterol (OR ≈ 2.04), and albumin (OR ≈ 1.61) are associated with higher severity, whereas age (OR ≈ 0.69), total protein (OR ≈ 0.63), creatinine (OR ≈ 0.61), bilirubin (OR ≈ 0.42), and AST (OR ≈ 0.42) are associated with lower severity. ALT shows the strongest effect, highlighting its importance, while other liver biomarkers also contribute to predicting Hepatitis C severity.

Now test the model using the Test set.

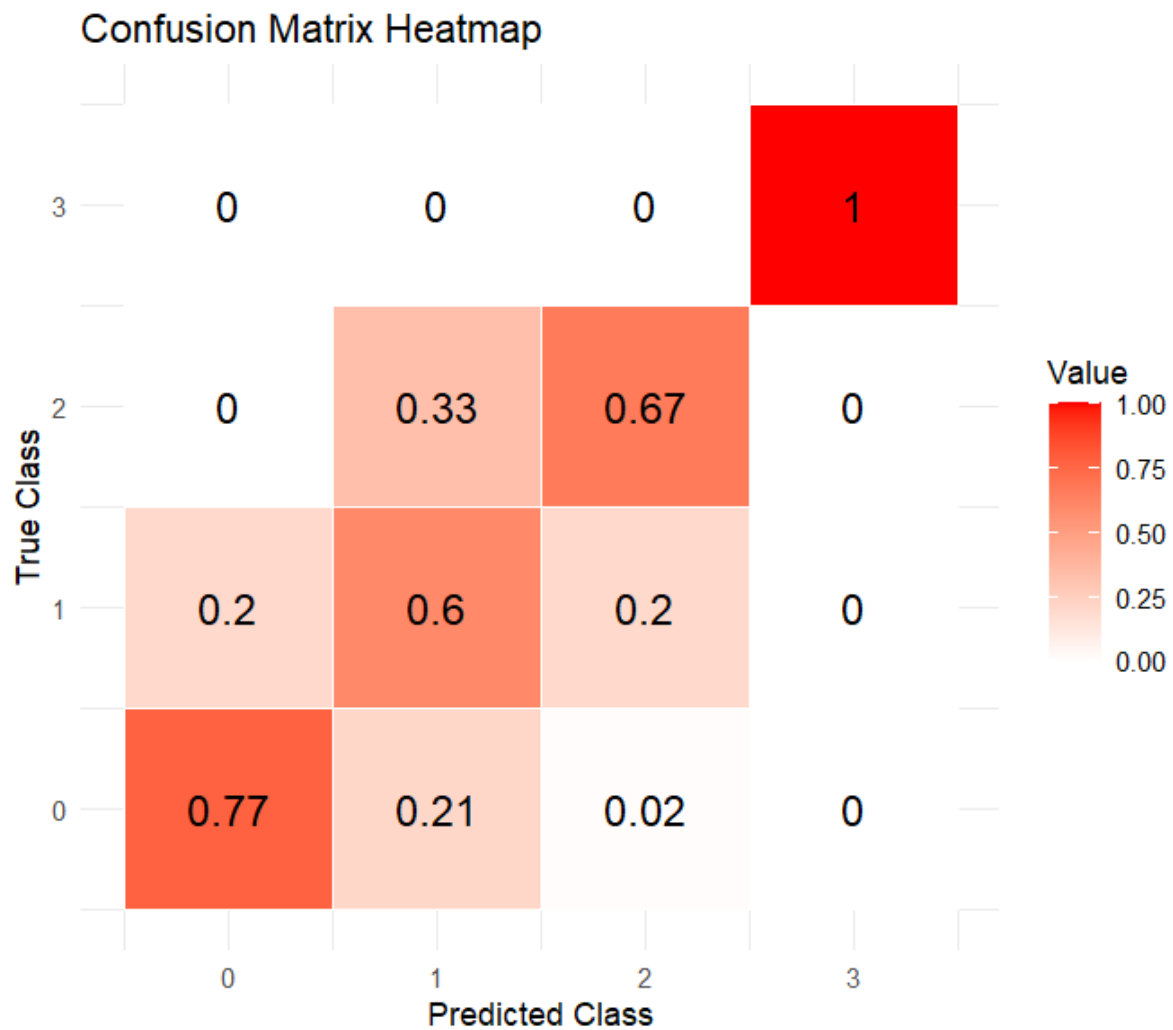
```
> prob_test <- predict(model_fixed, newdata = X_ts, type = "response")
> pred_test <- apply(prob_test, 1, which.max) - 1
> table(True = X_ts$Category_num, Predicted = pred_test)
      Predicted
True   0    1    2    3
  0 103   28    2    0
  1    1    3    1    0
  2    0    1    2    0
  3    0    0    0    6
> mean(pred_test == X_ts$Category_num)
[1] 0.7755102

> sensitivity <- diag(tab) / rowSums(tab)
> sensitivity
      0      1      2      3
0.7744361 0.6000000 0.6666667 1.0000000
> specificity <- sapply(1:nrow(tab), function(i) {
+   TP <- tab[i, i]
+   FN <- sum(tab[i, ]) - TP
+   FP <- sum(tab[, i]) - TP
+   TN <- sum(tab) - TP - FN - FP
+   TN / (TN + FP)
+ })
> specificity
[1] 0.9285714 0.7957746 0.9791667 1.0000000
```

Therefore, the accuracy of the fitted model is 77.55%, and the model shows strong performance in identifying healthy individuals (Class 0, sensitivity ≈ 0.77) and advanced

disease (Class 3, sensitivity ≈ 1), with moderate performance for fibrosis (Class 2, ≈ 0.67). However, sensitivity is low for early-stage Hepatitis C (Class 1, ≈ 0.60), indicating difficulty in detecting intermediate cases. Specificity is high across all classes (≈ 0.79 – 1), suggesting the model effectively avoids false positives, especially for severe disease categories.

3.3 Confusion Heat Matrix



Running this entire code 1000 times we found out the following:

```

> # Results (Here acc is the vector consisting accuracy in each run)
> mean(acc)    # average accuracy
[1] 0.8158095
> sd(acc)      # variability
[1] 0.04966337
> min(acc)     # worst case
[1] 0.6394558
> max(acc)     # best case
[1] 0.9455782

> colMeans(sens_all)
[1] 0.8286767 0.5274000 0.6300000 0.8638333
> colMeans(spec_all)
[1] 0.9195714 0.8415141 0.9790486 0.9969645

```

Thus, on an average, our model has 81.58% accuracy with a standard deviation of 0.049. The observed accuracies ranged from 63.94% to approximately 94.55%, indicating moderate variability across different data splits. The model also has a balanced accuracy of 71.24%, indicating good overall performance while accounting for class imbalance, though performance varies across minority classes.

Sensitivity is high for classes 0 and 3 (≈ 0.83 and ≈ 0.86), indicating that these classes are detected reliably. However, performance drops for class 1, with sensitivities around 0.53, suggesting difficulty in correctly identifying early stage of disease.

On the other hand, specificity remains consistently high across all classes (≈ 0.84 – 0.99), indicating that the model is very effective at correctly rejecting non-members of each class. Overall, this suggests that while the model is conservative and avoids false positives, it struggles with distinguishing minority or clinically similar classes, particularly class 1.

4 Conclusion

Since the response variable is ordinal in nature, we employed the proportional odds ordinal logistic regression model, which appropriately accounts for the inherent ordering in disease severity.

To address the severe class imbalance in the dataset, we incorporated inverse-frequency class weights into the likelihood function. This adjustment ensured that rare but clinically important disease stages, such as Fibrosis and Cirrhosis, were not overshadowed by the majority class (Blood Donors). The weighted ordinal model was fitted using the VGAM package in R under the assumption of proportional odds, implying that the effect of predictors remains constant across all cumulative logits.

The final selected model demonstrated strong predictive performance across multiple data splits. On the test set, the model achieved an average accuracy of approximately 81.58% and balanced accuracy of 71.24%, with consistently stable performance over repeated simulations (mean accuracy $\approx$ 81.58% with low variability). Importantly, the model also achieved reasonable sensitivity and specificity for minority classes, indicating that it is not biased solely toward the majority class, though it is difficult to detect early stages of disease (Specifically class 1).

From a clinical perspective, several biomarkers such as ALT, AST, Bilirubin, Creatinine, and Albumin were found to be statistically significant predictors of disease progression, aligning well with medical understanding of liver function deterioration. While the model identifies statistical associations, some results are not fully consistent with established clinical knowledge and should therefore be interpreted with caution.

5 Complete R Code

```
#----library needed----
library(ggplot2)
library(caTools)
library(VGAM)
library(dplyr)
#----setting dataset----

setwd("C:/Users/aparu/OneDrive/Desktop/Hepatitis C")
raw_data <- read.csv("hcvdat0.csv")

data <- na.omit(raw_data)
data <- data[ , -c(1,4)]
n_runs <- 1000
acc <- numeric(n_runs)
sens_all <- matrix(0, nrow = n_runs, ncol = 4)
spec_all <- matrix(0, nrow = n_runs, ncol = 4)

#deal with categories

table(data$Category)
prop.table(table(data$Category))

data$Category[data$Category=="0=suspect Blood Donor"]<-"0=Blood Donor"
data$Category_num <- as.numeric(factor(data$Category,
                                     levels = c("0=Blood Donor",
                                     "1=Hepatitis",
                                     "2=Fibrosis",
                                     "3=Cirrhosis")) - 1

data<- data[ , -1]
data[ , -12]<- scale(data[ , -12])
```

```

for(i in 1:n_runs){
  #set.seed(216)
  split <- sample.split(data$Category_num, SplitRatio = 0.75)
  X_tr <- subset(data, split == "TRUE")
  X_ts <- subset(data, split == "FALSE")
  #----fitting model----
  class_counts <- table(X_tr$Category_num)
  wts <- as.numeric(1/ class_counts[as.character(X_tr$Category_num)])
  wts <- wts / mean(wts)

  model <- vglm(Category_num ~ Age + ALB + ALT + AST + BIL +
                CHE + CHOL + CREA + GGT + ALP + PROT,
                family = cumulative(parallel = TRUE),
                data = X_tr,
                weights = wts)

  model_fixed <- vglm(Category_num ~ Age+ ALB + ALT + AST + BIL
                    + CREA + PROT + CHOL ,
                    family = cumulative(parallel = TRUE),
                    data = X_tr,
                    weights = wts)

  prob <- predict(model, newdata = X_tr, type = "response")
  pred <- apply(prob, 1, which.max) - 1
  table(True = X_tr$Category_num, Predicted = pred)
  mean(pred == X_tr$Category_num)

  prob_new <- predict(model_fixed, newdata = X_tr, type = "response")
  pred_new <- apply(prob_new, 1, which.max) - 1
  table(True = X_tr$Category_num, Predicted = pred_new)
  mean(pred_new == X_tr$Category_num)

```

```

#----test the model----

prob_test <- predict(model_fixed, newdata = X_ts, type = "response")
pred_test <- apply(prob_test, 1, which.max) - 1
table(True = X_ts$Category_num, Predicted = pred_test)
mean(pred_test == X_ts$Category_num)

cm <- table(
  factor(X_ts$Category_num, levels = 0:3),
  factor(pred_test, levels = 0:3)
)
# ensure all classes exist
cm <- as.matrix(cm)
classes <- 0:3

for(k in classes){

  TP <- cm[as.character(k), as.character(k)]

  FN <- sum(cm[as.character(k), ]) - TP
  FP <- sum(cm[, as.character(k)]) - TP
  TN <- sum(cm) - TP - FN - FP

  sens_all[i, k+1] <- TP / (TP + FN)
  spec_all[i, k+1] <- TN / (TN + FP)
}

# Accuracy
acc[i] <- mean(pred_test == X_ts$Category_num)

}

```



```
# Results
mean(acc)    # average accuracy
sd(acc)      # variability
min(acc)     # worst case
max(acc)     # best case

colMeans(sens_all)
colMeans(spec_all)
```

Git-hub-link - <https://github.com/Anushruta06/Hepatitis-c/blob/main/hepatitis-c.R>

References

- 1) A weighted-likelihood framework for class imbalance in Bayesian prediction models | Stanley E. Lazic.
- 2) Multilevel Ordinal Logit Models: A Proportional Odds Application Using Data from Brazilian Higher Education Institutions
- 3) Class-Weighted Classification: Trade-offs and Robust Approaches
- 4) Combining a class-weighted algorithm and machine learning models in landslide susceptibility mapping: A case study of Wanzhou section of the Three Gorges Reservoir, China
- 5) Class-Weighted Convolutional Features for Visual Instance Search
- 6) A Machine Learning Based Diabetes Mellitus Classification on Weighted Class using Inverse Class Frequency Method
- 7) An Instance Weighting Method To Induced Cost-Sensitive Trees